

Tugas Pemrograman Protokol MQTT Cyber-Physical Systems

Implementasi Komunikasi MQTT Menggunakan Python dan Mosquitto Broker



Disusun Oleh:

Kelas A Teknik Komputer

1. Aqilah Akma

[235150301111017]

**FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA
MALANG
2026**

DAFTAR ISI

DAFTAR ISI.....	2
A. PENDAHULUAN.....	3
B. DESAIN SISTEM.....	4
1. Arsitektur Cyber Physical System.....	4
2. Alur Jalur Komunikasi Data.....	5
C. IMPLEMENTASI PROGRAM SKENARIO.....	5
1. Skenario 1 : implementasi Dasar MQTT (Basic Publisher & Subscriber).....	6
Implementasi ini bertujuan untuk memvalidasi konektivitas dasar protokol MQTT dalam mengirimkan satu data telemetri berupa suhu ruangan tanpa retensi pesan. Komunikasi dilakukan melalui jalur topik smartroom/room1/temperature.....	6
a. publisher_basic.py.....	6
b. subscriber_basic.py.....	6
2. Skenario 2 : Kualitas Layanan (Quality of Service - QoS).....	7
a. publisher_qos.py.....	7
b. subscriber_qos.py.....	8
3. Skenario 3 : Penggunaan Beberapa Topik (Multi-Topic).....	9
a. publisher_multitopic.py.....	9
b. subscriber_multitopic.py.....	10
4. Skenario 4 : Penyaringan Topik Menggunakan Wildcard Plus (+).....	11
a. publisher_wildcard_plus.py.....	11
b. subscriber_wildcard_plus.py.....	12
5. Skenario 5 : Pemantauan Global Menggunakan Wildcard Hash (#).....	12
a. publisher_wildcard_hash.py.....	13
b. subscriber_wildcard_hash.py.....	14
D. PENGUJIAN DAN HASIL PROGRAM.....	15
1. Pengujian skenario 1: Komunikasi Dasar (Basic Publisher & Subscriber).....	15
2. Pengujian skenario 2 : Kualitas Layanan (QoS 0, 1, dan 2).....	15
3. Pengujian skenario 3 : Penggunaan Beberapa Topik (Multi-Topic).....	16
4. Pengujian skenario 4 : Penyaringan Topik dengan Wildcard Plus (+).....	16
5. Pengujian Skenario 5: Pemantauan Global dengan Wildcard Hash (#).....	17
E. ANALISIS HASIL PENGUJIAN.....	19
F. KESIMPULAN.....	20

A. PENDAHULUAN

Cyber Physical System (CPS) merupakan sistem cerdas yang mengintegrasikan komponen komputasi siber dengan proses fisik melalui mekanisme pemantauan, komunikasi, dan pengendalian secara berkelanjutan. Sistem ini memanfaatkan sensor untuk menyerap data dari lingkungan fisik, mengolah informasi pada ruang digital, dan mengeksekusi tindakan balik yang adaptif. Dalam ekosistem Jaringan Internet of Things (IoT) yang menopang CPS, protokol Message Queuing Telemetry Transport (MQTT) menjadi standar komunikasi paling andal karena menerapkan arsitektur publish-subscribe yang sangat ringan, efisien, dan mampu memisahkan dependensi jaringan antara pengirim (publisher) dan penerima data (subscriber) melalui entitas penengah (broker).

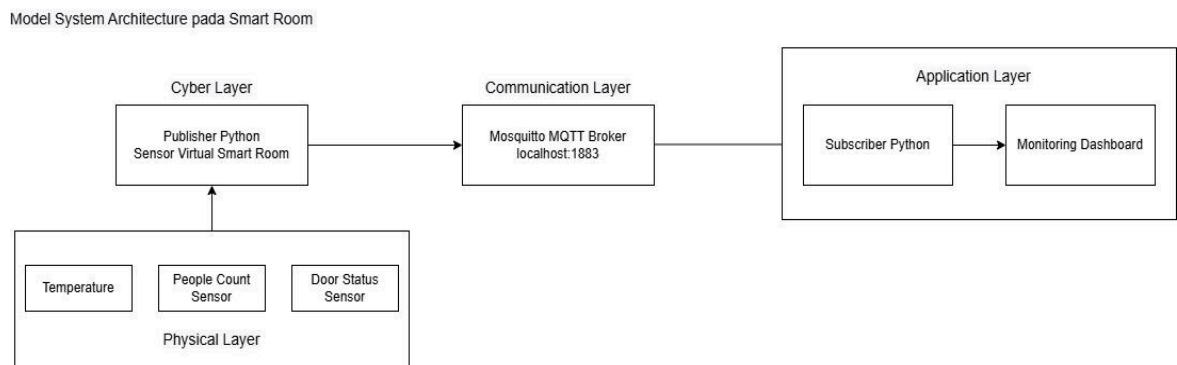
Praktikum ini menerapkan protokol MQTT untuk membangun simulasi sistem Smart Room Monitoring berbasis bahasa pemrograman Python dan Eclipse Mosquitto Broker. Implementasi program mencakup pengiriman telemetri data sensor virtual berupa parameter suhu, kelembapan udara, dan intensitas cahaya secara berulang melalui beberapa topik terstruktur. Selain menguji komunikasi data dasar, rangkaian pengujian juga dilakukan dengan memvariasikan tingkat Quality of Service (QoS 0, 1, dan 2) serta mengimplementasikan teknik filtrasi topik menggunakan wildcard (+ dan #) untuk menganalisis karakteristik distribusi pesan pada berbagai kondisi saluran komunikasi.

Tujuan utama dari pelaksanaan praktikum ini adalah untuk memahami secara komprehensif konsep arsitektur komunikasi data MQTT di dalam ranah Cyber Physical System. Melalui pemenuhan tugas ini, mahasiswa diharapkan mampu mengonfigurasi Mosquitto Broker secara mandiri, mengimplementasikan skrip kode publisher dan subscriber menggunakan pustaka Python secara fungsional, serta menganalisis pengaruh variasi tingkat jaminan QoS dan hierarki penamaan topik terhadap efisiensi proses transmisi paket data siber-fisik.

B. DESAIN SISTEM

Desain sistem pada proyek Smart Room Monitoring System ini dirancang menggunakan pendekatan arsitektur Cyber Physical System (CPS) untuk memantau kondisi ruangan secara aktual dan dinamis. Arsitektur ini mengintegrasikan komponen fisik berupa perangkat sensor telemetri dengan komponen komputasi digital sebagai pengendali dan penampil data.

1. Arsitektur Cyber Physical System



Gambar 1. Model Komunikasi MQTT pada Smart Room Monitoring

Secara fungsional, sistem smartroom ini dibagi ke dalam 4 arsitektur utama yang saling terintegrasi:

- **Physical Layer** ini adalah lapisan paling dasar yang mencakup komponen fisik berupa sensor penangkap data lingkungan nyata, yaitu Temperature Sensor, People Count Sensor, dan Door Status Sensor.
- **Cyber Layer** adalah lapisan komputasi yang bertindak sebagai sensor virtual. Lapisan ini diimplementasikan menggunakan skrip kode Python (Publisher Python Sensor Virtual Smart Room) untuk mengolah input data dari Physical Layer sebelum dikirimkan ke jaringan.
- **Communication Layer** adalah lapisan gerbang komunikasi data yang akan menjembatani pengiriman pesan. Lapisan ini menggunakan Mosquitto MQTT Broker yang berjalan pada server lokal (localhost:1883) dengan protokol MQTT.
- **Application Layer** adalah lapisan teratas penampil data yang terdiri dari skrip Subscriber Python untuk menangkap pesan dari broker, serta Monitoring

Dashboard sebagai antarmuka pemantauan data ruangan secara real-time bagi user.

2. Alur Jalur Komunikasi Data

Mekanisme sirkulasi data dari hulu ke hilir pada sistem ini dijabarkan melalui langkah-langkah terstruktur berikut::

- a. Komponen pada Physical Layer mendeteksi perubahan kondisi lingkungan ruangan.
- b. Data dari sensor dikirimkan ke Cyber Layer untuk dikemas menjadi pesan (payload) melalui skrip Publisher Python.
- c. Skrip Publisher mengirimkan data tersebut ke Communication Layer (Mosquitto MQTT Broker pada port 1883) menggunakan jalur topik (topic path) yang spesifik.
- d. Broker menyalurkan data secara instan kepada skrip Subscriber yang berada di Application Layer.
- e. Lapisan aplikasi menangkap data tersebut, mem-parsing isinya, lalu menampilkannya pada Monitoring Dashboard sebagai informasi pemantauan real-time.

C. IMPLEMENTASI PROGRAM SKENARIO

Implementasi sistem ini dilakukan menggunakan bahasa pemrograman Python dengan bantuan library paho-mqtt serta Mosquitto sebagai MQTT broker. Program yang dikembangkan terdiri dari beberapa skrip publisher dan subscriber di dalam folder code untuk memenuhi lima skenario pengujian. Skenario tersebut meliputi komunikasi dasar publisher subscriber, pengujian Quality of Service (QoS), penggunaan beberapa topik, wildcard +, dan wildcard #.

Sebelum program dijalankan, dilakukan instalasi Python, Mosquitto Broker, serta library paho-mqtt. Mosquitto disini digunakan sebagai message broker yang akan menerima dan mendistribusikan pesan MQTT, sedangkan library paho-mqtt memungkinkan program Python melakukan komunikasi menggunakan protokol MQTT dan Instalasi librarynya dilakukan menggunakan perintah berikut pada terminal VSCode

```
pip install paho-mqtt
```

Broker Mosquitto dijalankan pada localhost dengan port default MQTT yaitu 1883. Seluruh kode program diletakkan secara terstruktur di dalam folder code agar sistem dapat mengidentifikasi skrip publisher dan subscriber dengan rapi.

1. Skenario 1 : implementasi Dasar MQTT (Basic Publisher & Subscriber)

Implementasi ini bertujuan untuk memvalidasi konektivitas dasar protokol MQTT dalam mengirimkan satu data telemetry berupa suhu ruangan tanpa retensi pesan. Komunikasi dilakukan melalui jalur topik smartroom/room1/temperature.

a. publisher_basic.py

```
import paho.mqtt.client as mqtt
import time

client =
mqtt.Client(callback_api_version=mqtt.CallbackAPIVersion.VERSION
2)
print("[INFO] Menghubungkan ke Mosquitto Broker lokal...")
client.connect("localhost", 1883)
client.loop_start()

try:
    while True:
        val_temp = "26.5"
        topic = "smartroom/room1/temperature"
        print(f"[PUBLISH] Mengirim data suhu ke {topic} ->
{val_temp}°C")
        client.publish(topic, payload=val_temp, qos=0)
        time.sleep(5)
except KeyboardInterrupt:
    print("\n[INFO] Memutus koneksi publisher...")
```

b. subscriber_basic.py

```
import paho.mqtt.client as mqtt

def on_connect(client, userdata, flags, rc, properties=None):
    print("[INFO] Berhasil terhubung ke broker Mosquitto.")
```

```
print("[SUBSCRIBE] Menunggu kiriman data dari topik  
suhu...\n" + "-"*60)  
client.subscribe("smartroom/room1/temperature")  
  
def on_message(client, userdata, msg):  
    payload_data = msg.payload.decode()  
    print(f"[SUBSCRIBE] Data diterima dari {msg.topic} -> Suhu:  
{payload_data}°C")  
  
client =  
mqtt.Client(callback_api_version=mqtt.CallbackAPIVersion.VERSION  
2)  
client.on_connect = on_connect  
client.on_message = on_message  
  
client.connect("localhost", 1883)  
client.loop_forever()
```

2. Skenario 2 : Kualitas Layanan (Quality of Service - QoS)

Skenario ini bertujuan untuk menganalisis perilaku pengiriman pesan berdasarkan tiga tingkat keandalan yang berbeda, yaitu QoS 0, QoS 1, dan QoS 2 melalui topik smartroom/room1/qos.

a. publisher_qos.py

```
import paho.mqtt.client as mqtt  
import time  
import random  
  
client =  
mqtt.Client(callback_api_version=mqtt.CallbackAPIVersion.VERSIO  
N2)  
client.connect("localhost", 1883)  
client.loop_start()  
  
try:  
    while True:  
        print("\n== MEMULAI SIKLUS PENGUJIAN AKURASI DATA
```

```
(QoS) ===")
    # Nilai diacak di dalam loop agar berubah di setiap
    baris cetakan
    temp = round(random.uniform(24.0, 29.0), 1)

    # Masing-masing data dikirim ke topik
    'smartroom/room1/qos_test'
    print(f"[QoS 0 Sent] Mengirim telemetri suhu ->
    {temp}°C")
    client.publish("smartroom/room1/qos_test",
    payload=f"Suhu: {temp}°C", qos=0)
    time.sleep(2)

    print(f"[QoS 1 Sent] Mengirim data jumlah orang -> 5
    orang")
    client.publish("smartroom/room1/qos_test",
    payload="Orang: 5", qos=1)
    time.sleep(2)

    print(f"[QoS 2 Sent] Mengirim status akses pintu ->
    CLOSED")
    client.publish("smartroom/room1/qos_test",
    payload="Pintu: CLOSED", qos=2)

    time.sleep(5)
except KeyboardInterrupt:
    print("\n[INFO] Menghentikan simulasi pengujian QoS...")
```

b. subscriber_qos.py

```
import paho.mqtt.client as mqtt

def on_connect(client, userdata, flags, rc, properties=None):
    print("[INFO] Mengaktifkan filter jaminan paket data
    ruangan (QoS 0, 1, 2).\n" + "-"*60)
    client.subscribe("smartroom/room1/qos_test", qos=2)

def on_message(client, userdata, msg):
    payload_data = msg.payload.decode()
    print(f"[TERIMA - QoS {msg.qos}] Data: {payload_data}")
```



```
client =  
mqtt.Client(callback_api_version=mqtt.CallbackAPIVersion.VERSION2)  
client.on_connect = on_connect  
client.on_message = on_message  
  
client.connect("localhost", 1883)  
client.loop_forever()
```

3. Skenario 3 : Penggunaan Beberapa Topik (Multi-Topic)

Pengujian ini dilakukan untuk mensimulasikan kondisi nyata kamar pintar yang memiliki banyak sensor independen. Tiga topik berbeda digunakan secara simultan, yaitu smartroom/room1/temperature (suhu), smartroom/room1/people (jumlah orang), dan smartroom/room1/door (status pintu).

a. publisher_multitopic.py

```
import paho.mqtt.client as mqtt  
import time  
import random  
  
client =  
mqtt.Client(callback_api_version=mqtt.CallbackAPIVersion.VERSION2)  
client.connect("localhost", 1883)  
client.loop_start()  
  
try:  
    while True:  
        # Nilai dibuat acak di dalam loop agar terminal  
        bergerak dinamis dan realistis  
        temp = round(random.uniform(22.0, 30.0), 1)  
        people = random.randint(1, 15)  
        door = random.choice(["OPEN", "CLOSED"])  
  
        print(f"\n[KIRIM MULTI-TOPIK] Mendistribusikan data  
sensor:")
```

```

        print(f" -> smartroom/room1/temperature : {temp}°C")
        client.publish("smartroom/room1/temperature",
payload=f"{temp}", qos=0)
        time.sleep(1)

        print(f" -> smartroom/room1/people      : {people}
Orang")
        client.publish("smartroom/room1/people",
payload=f"{people}", qos=0)
        time.sleep(1)

        print(f" -> smartroom/room1/door        : {door}")
        client.publish("smartroom/room1/door", payload=door,
qos=0)

        time.sleep(5)
except KeyboardInterrupt:
    print("\n[INFO] Menghentikan pengiriman multi-topik...")

```

b. subscriber_multitopic.py

```

import paho.mqtt.client as mqtt

def on_connect(client, userdata, flags, rc, properties=None):
    print("[INFO] Mendaftarkan multi-topik paralel secara
terpisah...\n" + "-"*60)
    client.subscribe("smartroom/room1/temperature")
    client.subscribe("smartroom/room1/people")
    client.subscribe("smartroom/room1/door")

def on_message(client, userdata, msg):
    payload_data = msg.payload.decode()
    print(f"[DATA MASUK] Alamat Topik: {msg.topic} -> Nilai:
{payload_data}")

client =
mqtt.Client(callback_api_version=mqtt.CallbackAPIVersion.VERSION2)
client.on_connect = on_connect
client.on_message = on_message

```

```
client.connect("localhost", 1883)
client.loop_forever()
```

4. Skenario 4 : Penyaringan Topik Menggunakan Wildcard Plus (+)

Skenario ini menerapkan teknik penyaringan satu tingkat (single-level wildcard) untuk memantau data sejenis dari beberapa ruangan yang berbeda secara efisien tanpa perlu mendaftarkan topik setiap ruangan secara manual. Topik filter yang didaftarkan oleh pihak penerima pesan adalah smartroom/+temperature.

a. publisher_wildcard_plus.py

```
import paho.mqtt.client as mqtt
import time
import random

client =
mqtt.Client(callback_api_version=mqtt.CallbackAPIVersion.VERSION2)
client.connect("localhost", 1883)
client.loop_start()

try:
    while True:
        temp_r1 = round(random.uniform(23.0, 26.0), 1)
        temp_r2 = round(random.uniform(24.0, 27.0), 1)

        print(f"\n[PUBLISH SKENARIO 4] Menyebarkan data ke broker:")
        print(f" -> Kirim ke smartroom/room1/temperature : {temp_r1}")
        client.publish("smartroom/room1/temperature",
payload=str(temp_r1), qos=0)
        time.sleep(1)

        print(" -> Kirim ke smartroom/room1/door : OPEN (Harusnya terblokir di Subscriber)")
        client.publish("smartroom/room1/door", payload="OPEN",
```

```
qos=0)

    time.sleep(1)

    print(f" -> Kirim ke smartroom/room2/temperature :
{temp_r2}")
    client.publish("smartroom/room2/temperature",
payload=str(temp_r2), qos=0)

    time.sleep(5)
except KeyboardInterrupt:
    print("\n[INFO] Menghentikan pengisian data wildcard
plus...")
```

b. subscriber_wildcard_plus.py

```
import paho.mqtt.client as mqtt

def on_connect(client, userdata, flags, rc, properties=None):
    print("[INFO] Menghubungkan ke broker Mosquitto.")
    print("[INFO] Menunggu data dengan filter satu tingkat:
smartroom/+ /temperature\n" + "-"*60)
    client.subscribe("smartroom/+ /temperature")

def on_message(client, userdata, msg):
    payload_data = msg.payload.decode()
    print(f"[FILTER + MATCH] Jalur: {msg.topic} -> Nilai
Terbaca: {payload_data}°C")

client =
mqtt.Client(callback_api_version=mqtt.CallbackAPIVersion.VERSION2)
client.on_connect = on_connect
client.on_message = on_message

client.connect("localhost", 1883)
client.loop_forever()
```

5. Skenario 5 : Pemantauan Global Menggunakan Wildcard Hash (#)

Skenario terakhir ini bertujuan untuk membangun sistem pusat pemantauan (global logging) yang mampu menangkap seluruh aktivitas data telemetri secara menyeluruh dari semua tingkatan hierarki topik di bawah struktur utama menggunakan filter multi-tingkat smartroom/#.

a. publisher_wildcard_hash.py

```
import paho.mqtt.client as mqtt
import time
import random

client =
mqtt.Client(callback_api_version=mqtt.CallbackAPIVersion.VERSION2)
client.connect("localhost", 1883)
client.loop_start()

try:
    while True:
        temp = round(random.uniform(21.0, 25.0), 1)
        people = random.randint(1, 10)
        door = random.choice(["OPEN", "CLOSED"])

        print(f"\n[PUBLISH GLOBAL] Menyiarkan seluruh aktivitas sensor ruangan:")
        print(f" -> smartroom/room1/temperature : {temp}")
        client.publish("smartroom/room1/temperature",
payload=f"{temp}°C", qos=0)
        time.sleep(1)

        print(f" -> smartroom/room1/people      : {people}")
        client.publish("smartroom/room1/people",
payload=f"{people} Orang", qos=0)
        time.sleep(1)

        print(f" -> smartroom/room1/door          : {door}")
        client.publish("smartroom/room1/door", payload=door,
qos=0)
```

```
        time.sleep(5)
except KeyboardInterrupt:
    print("\n[INFO] Menutup sistem sebaran data global...")
```

b. subscriber_wildcard_hash.py

```
import paho.mqtt.client as mqtt

def on_connect(client, userdata, flags, rc, properties=None):
    print("[INFO] Mengaktifkan pusat pemantauan/Logger Global: smartroom/#\n" + "-"*60)
    client.subscribe("smartroom/#")

def on_message(client, userdata, msg):
    payload_data = msg.payload.decode()
    print(f"[LOG GLOBAL] Topik Asal: {msg.topic} | Payload Data: {payload_data}")

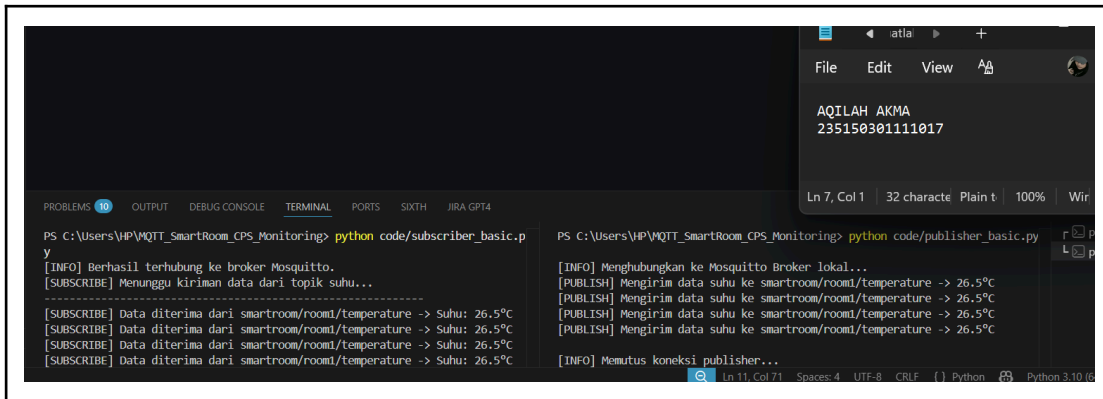
client =
mqtt.Client(callback_api_version=mqtt.CallbackAPIVersion.VERSION2)
client.on_connect = on_connect
client.on_message = on_message

client.connect("localhost", 1883)
client.loop_forever()
```

D. PENGUJIAN DAN HASIL PROGRAM

1. Pengujian skenario 1: Komunikasi Dasar (Basic Publisher & Subscriber)

Langkah pengujian dilakukan dengan menjalankan skrip penerima data tunggal melalui perintah python code/subscriber_basic.py pada terminal kiri. Kemudian, skrip pengirim data utama dijalankan melalui perintah python code/publisher_basic.py pada terminal sebelah kanan.



```

PS C:\Users\VP\WQTT_SmartRoom_CPS_Monitoring> python code/subscriber_basic.py
[INFO] Berhasil terhubung ke broker Mosquitto.
[SUBSCRIBE] Menunggu kiriman data dari topik suhu...
-----
[SUBSCRIBE] Data diterima dari smartroom/room1/temperature -> Suhu: 26.5°C
[SUBSCRIBE] Data diterima dari smartroom/room1/temperature -> Suhu: 26.5°C
[SUBSCRIBE] Data diterima dari smartroom/room1/temperature -> Suhu: 26.5°C
[SUBSCRIBE] Data diterima dari smartroom/room1/temperature -> Suhu: 26.5°C

PS C:\Users\VP\WQTT_SmartRoom_CPS_Monitoring> python code/publisher_basic.py
[INFO] Menghubungkan ke Mosquitto Broker lokal...
[PUBLISH] Mengirim data suhu ke smartroom/room1/temperature -> 26.5°C
[PUBLISH] Mengirim data suhu ke smartroom/room1/temperature -> 26.5°C
[PUBLISH] Mengirim data suhu ke smartroom/room1/temperature -> 26.5°C
[PUBLISH] Mengirim data suhu ke smartroom/room1/temperature -> 26.5°C
[INFO] Memutus koneksi publisher...
  
```

Output pengujian ini menunjukkan terminal publisher berhasil mengirimkan data suhu konstan sebesar 26.5°C secara konsisten ke topik smartroom/room1/temperature. Terminal subscriber secara real-time menangkap data tersebut tanpa adanya data yang terputus, membuktikan koneksi TCP mendasar antara klien dan broker Mosquitto telah sukses dikonfigurasi.

2. Pengujian skenario 2 : Kualitas Layanan (QoS 0, 1, dan 2)

Langkah pengujian dilakukan dengan menjalankan skrip penerima jaminan paket melalui perintah python code/subscriber_qos.py pada terminal kiri. Kemudian, skrip pengirim simulasi kualitas layanan dijalankan melalui perintah python code/publisher_qos.py pada terminal kanan.



```

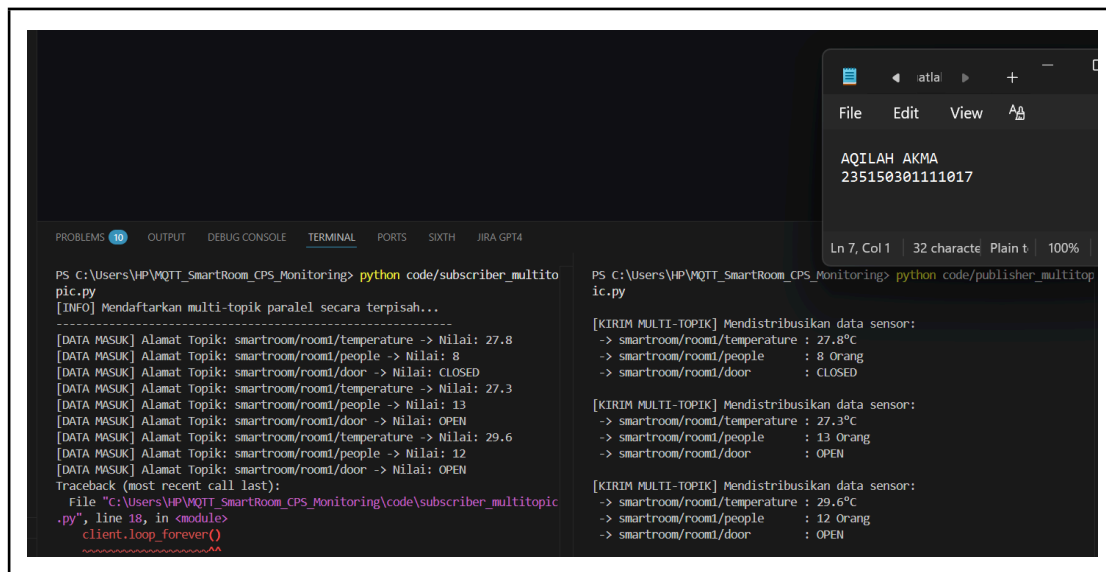
KeyboardInterrupt
PS C:\Users\VP\WQTT_SmartRoom_CPS_Monitoring> python code/subscriber_qos.py
[INFO] Mengaktifkan filter jaminan paket data ruangan (QoS 0, 1, 2).
-----
[TERIMA - QoS 0] Data: Suhu: 27.8°C
[TERIMA - QoS 1] Data: Orang: 5
[TERIMA - QoS 2] Data: Pintu: CLOSED

PS C:\Users\VP\WQTT_SmartRoom_CPS_Monitoring> python code/publisher_qos.py
=== MEMULAI SIKLUS PENGUJIAN AKURASI DATA (QoS) ===
[QoS 0 Sent] Mengirim telemetry suhu -> 27.8°C
[QoS 1 Sent] Mengirim data jumlah orang -> 5 orang
[QoS 2 Sent] Mengirim status akses pintu -> CLOSED
  
```

Output pengujian ini menunjukkan bahwa sistem berhasil menangani tiga tingkat kualitas layanan (Quality of Service) yang berbeda pada satu siklus perulangan. Terminal subscriber mencetak pesan dengan rincian identifikasi Level Jaminan QoS: 0 pengiriman cepat tanpa verifikasi, Level Jaminan QoS: 1 menjamin pesan sampai minimal sekali dengan acknowledgment, dan Level Jaminan QoS: 2 menjamin keandalan data mutlak melalui jabat tangan empat arah secara berurutan tanpa terjadi tumpang tindih struktur pesan yang berjalan.

3. Pengujian skenario 3 : Penggunaan Beberapa Topik (Multi-Topic)

Langkah pengujian dilakukan dengan menjalankan skrip penerima multi-saluran melalui perintah python code/subscriber_multitopic.py pada terminal kiri. Kemudian, skrip pengirim distribusi parameter dijalankan melalui perintah python code/publisher_multitopic.py pada terminal kanan.



```

PS C:\Users\WP\WQTT_SmartRoom_CPS_Monitoring> python code/subscriber_multitopic.py
[INFO] Mendaftarkan multi-topik paralel secara terpisah...
-----
[DATA MASUK] Alamat Topik: smartroom/room1/temperature -> Nilai: 27.8
[DATA MASUK] Alamat Topik: smartroom/room1/people -> Nilai: 8
[DATA MASUK] Alamat Topik: smartroom/room1/door -> Nilai: CLOSED
[DATA MASUK] Alamat Topik: smartroom/room1/temperature -> Nilai: 27.3
[DATA MASUK] Alamat Topik: smartroom/room1/people -> Nilai: 13
[DATA MASUK] Alamat Topik: smartroom/room1/door -> Nilai: OPEN
[DATA MASUK] Alamat Topik: smartroom/room1/temperature -> Nilai: 29.6
[DATA MASUK] Alamat Topik: smartroom/room1/people -> Nilai: 12
[DATA MASUK] Alamat Topik: smartroom/room1/door -> Nilai: OPEN
Traceback (most recent call last):
  File "C:\Users\WP\WQTT_SmartRoom_CPS_Monitoring\code\subscriber_multitopic.py", line 18, in <module>
    client.loop_forever()
    ~~~~~^~~~~~

PS C:\Users\WP\WQTT_SmartRoom_CPS_Monitoring> python code/publisher_multitopic.py
[KIRIM MULTI-TOPIK] Mendistribusikan data sensor:
-> smartroom/room1/temperature : 27.8°C
-> smartroom/room1/people : 8 Orang
-> smartroom/room1/door : CLOSED

[KIRIM MULTI-TOPIK] Mendistribusikan data sensor:
-> smartroom/room1/temperature : 27.3°C
-> smartroom/room1/people : 13 Orang
-> smartroom/room1/door : OPEN

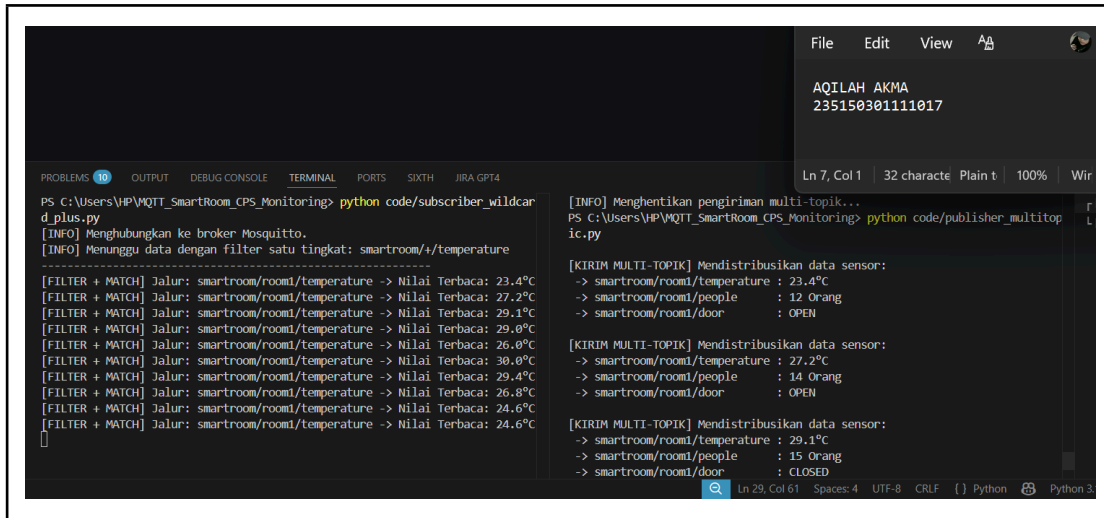
[KIRIM MULTI-TOPIK] Mendistribusikan data sensor:
-> smartroom/room1/temperature : 29.6°C
-> smartroom/room1/people : 12 Orang
-> smartroom/room1/door : OPEN
  
```

Output pengujian menampilkan terminal publisher mendistribusikan tiga tipe parameter fisik berbeda, yaitu suhu (temperature), jumlah hunian manusia (people), dan status keamanan akses pintu (door). Terminal subscriber secara paralel berhasil memilah pesan masuk berdasarkan saluran topiknya masing-masing dan mengekstrak nilai muatan (payload) secara akurat tanpa tertukar asek.

4. Pengujian skenario 4 : Penyaringan Topik dengan Wildcard Plus (+)

Langkah pengujian dilakukan dengan menjalankan skrip filter bertingkat melalui perintah python code/subscriber_wildcard_plus.py pada terminal kiri. Kemudian,

distribusi data acak dipicu dengan mengeksekusi skrip python code/publisher_multitopic.py pada terminal kanan.



```

File Edit View
AQILAH AKMA
235150301111017

PROBLEMS 10 OUTPUT DEBUG CONSOLE TERMINAL PORTS SIXTH JIRA GPT4
Ln 7, Col 1 | 32 character Plain t | 100% | Win

PS C:\Users\HP\WQTT_SmartRoom_CPS_Monitoring> python code/subscriber_wildcard_plus.py
[INFO] Menghubungkan ke broker Mosquitto.
[INFO] Menunggu data dengan filter satu tingkat: smartroom/+/temperature

[FILTER + MATCH] Jalur: smartroom/room1/temperature -> Nilai Terbaca: 23.4°C
[FILTER + MATCH] Jalur: smartroom/room1/temperature -> Nilai Terbaca: 27.2°C
[FILTER + MATCH] Jalur: smartroom/room1/temperature -> Nilai Terbaca: 29.1°C
[FILTER + MATCH] Jalur: smartroom/room1/temperature -> Nilai Terbaca: 29.0°C
[FILTER + MATCH] Jalur: smartroom/room1/temperature -> Nilai Terbaca: 26.0°C
[FILTER + MATCH] Jalur: smartroom/room1/temperature -> Nilai Terbaca: 30.0°C
[FILTER + MATCH] Jalur: smartroom/room1/temperature -> Nilai Terbaca: 29.4°C
[FILTER + MATCH] Jalur: smartroom/room1/temperature -> Nilai Terbaca: 26.8°C
[FILTER + MATCH] Jalur: smartroom/room1/temperature -> Nilai Terbaca: 24.6°C
[FILTER + MATCH] Jalur: smartroom/room1/temperature -> Nilai Terbaca: 24.6°C

[INFO] Menghentikan pengiriman multi-topik...
PS C:\Users\HP\WQTT_SmartRoom_CPS_Monitoring> python code/publisher_multitopic.py

[KIRIM MULTI-TOPIK] Mendistribusikan data sensor:
-> smartroom/room1/temperature : 23.4°C
-> smartroom/room1/people : 12 Orang
-> smartroom/room1/door : OPEN

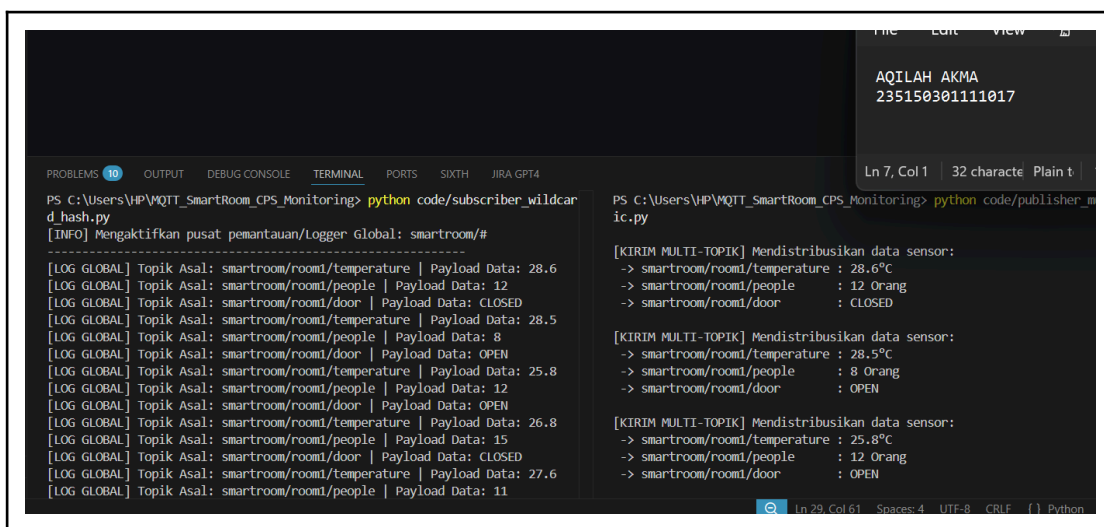
[KIRIM MULTI-TOPIK] Mendistribusikan data sensor:
-> smartroom/room1/temperature : 27.2°C
-> smartroom/room1/people : 14 Orang
-> smartroom/room1/door : OPEN

[KIRIM MULTI-TOPIK] Mendistribusikan data sensor:
-> smartroom/room1/temperature : 29.1°C
-> smartroom/room1/people : 15 Orang
-> smartroom/room1/door : CLOSED
  
```

Output pengujian ini membuktikan efektivitas fungsi Single-Level Wildcard (+). Terminal subscriber mendaftarkan filter topik smartroom/+/temperature. Ketika publisher mengirimkan data multi-sensor suhu, orang, dan pintu, terminal subscriber secara otomatis menyaring dan hanya mencetak parameter suhu saja dari ruangan terkait, sedangkan paket data di luar sub-topik suhu berhasil diabaikan secara aman.

5. Pengujian Skenario 5: Pemantauan Global dengan Wildcard Hash (#)

Langkah pengujian dilakukan dengan menjalankan skrip pengawas data global melalui perintah python code/subscriber_wildcard_hash.py pada terminal kiri. Selanjutnya, sebaran seluruh aktivitas sensor disimulasikan menggunakan perintah python code/publisher_multitopic.py pada terminal kanan.



```

File Edit View
AQILAH AKMA
235150301111017

PROBLEMS 10 OUTPUT DEBUG CONSOLE TERMINAL PORTS SIXTH JIRA GPT4
Ln 7, Col 1 | 32 character Plain t | 100% | Win

PS C:\Users\HP\WQTT_SmartRoom_CPS_Monitoring> python code/subscriber_wildcard_hash.py
[INFO] Mengaktifkan pusat pemantauan/Logger Global: smartroom/#

[LOG GLOBAL] Topik Asal: smartroom/room1/temperature | Payload Data: 28.6
[LOG GLOBAL] Topik Asal: smartroom/room1/people | Payload Data: 12
[LOG GLOBAL] Topik Asal: smartroom/room1/door | Payload Data: CLOSED
[LOG GLOBAL] Topik Asal: smartroom/room1/temperature | Payload Data: 28.5
[LOG GLOBAL] Topik Asal: smartroom/room1/people | Payload Data: 8
[LOG GLOBAL] Topik Asal: smartroom/room1/door | Payload Data: OPEN
[LOG GLOBAL] Topik Asal: smartroom/room1/temperature | Payload Data: 25.8
[LOG GLOBAL] Topik Asal: smartroom/room1/people | Payload Data: 12
[LOG GLOBAL] Topik Asal: smartroom/room1/door | Payload Data: OPEN
[LOG GLOBAL] Topik Asal: smartroom/room1/temperature | Payload Data: 26.8
[LOG GLOBAL] Topik Asal: smartroom/room1/people | Payload Data: 15
[LOG GLOBAL] Topik Asal: smartroom/room1/door | Payload Data: CLOSED
[LOG GLOBAL] Topik Asal: smartroom/room1/temperature | Payload Data: 27.6
[LOG GLOBAL] Topik Asal: smartroom/room1/people | Payload Data: 11

[INFO] Menghentikan pengiriman multi-topik...
PS C:\Users\HP\WQTT_SmartRoom_CPS_Monitoring> python code/publisher_multitopic.py

[KIRIM MULTI-TOPIK] Mendistribusikan data sensor:
-> smartroom/room1/temperature : 28.6°C
-> smartroom/room1/people : 12 Orang
-> smartroom/room1/door : CLOSED

[KIRIM MULTI-TOPIK] Mendistribusikan data sensor:
-> smartroom/room1/temperature : 28.5°C
-> smartroom/room1/people : 8 Orang
-> smartroom/room1/door : OPEN

[KIRIM MULTI-TOPIK] Mendistribusikan data sensor:
-> smartroom/room1/temperature : 25.8°C
-> smartroom/room1/people : 12 Orang
-> smartroom/room1/door : OPEN
  
```

Output pengujian ini membuktikan keberhasilan penerapan mekanisme Multi-Level Wildcard (#) sebagai Global Monitoring/Logger. Melalui penulisan sub-topik root smartroom/#, terminal subscriber mampu menangkap, membongkar, dan mencatat seluruh jenis transmisi data yang melewati broker Mosquitto baik data suhu, komponen orang, maupun status pintu tanpa perlu melakukan pendaftaran manual satu per satu.

E. ANALISIS HASIL PENGUJIAN

Berdasarkan hasil pengujian yang telah dilakukan, sistem komunikasi MQTT pada skenario Smart Room Monitoring ini mampu bekerja dengan baik dan stabil. Pengiriman data sensor melalui Mosquitto Broker berlangsung tanpa kendala, ditunjukkan dengan payload yang dapat diterima secara konsisten oleh subscriber. Hal ini menunjukkan bahwa mekanisme komunikasi berbasis publish-subscribe pada MQTT efektif digunakan untuk pertukaran data pemantauan ruangan secara real-time.

Selain itu, pengujian terhadap berbagai level Quality of Service (QoS) menunjukkan bahwa setiap tingkat QoS memiliki karakteristik yang berbeda sesuai kebutuhan sistem. QoS 0 menawarkan kecepatan pengiriman yang tinggi, sedangkan QoS 1 dan QoS 2 memberikan tingkat keandalan yang lebih baik melalui mekanisme konfirmasi pesan. Penggunaan wildcard (+ dan #) juga terbukti mempermudah proses pemantauan banyak sensor sekaligus karena subscriber dapat melakukan penyaringan maupun pencatatan seluruh data tanpa perlu berlangganan ke setiap topik secara terpisah. Hasil ini menunjukkan bahwa MQTT merupakan protokol yang fleksibel, efisien, dan mudah dikembangkan untuk implementasi sistem IoT yang lebih kompleks.

F. KESIMPULAN

Pada praktikum ini, implementasi protokol MQTT menggunakan bahasa pemrograman Python dan Mosquitto Broker dalam arsitektur Cyber Physical System (CPS) telah berhasil dilakukan. Hasil pengujian menunjukkan bahwa pola komunikasi publish-subscribe sangat cocok digunakan pada sistem pemantauan smart room secara real-time karena mampu bekerja dengan penggunaan daya dan bandwidth yang relatif rendah.

Selain itu, praktikum ini menunjukkan bahwa perancangan struktur topik yang baik serta pemilihan tingkat QoS yang sesuai memiliki peran penting dalam memastikan informasi dapat dikirim dan diterima dengan andal. Penggunaan fitur wildcard juga mempermudah pengelolaan data dari banyak sensor tanpa perlu menulis kode yang berulang. Dengan kelebihan tersebut, sistem yang dibangun memiliki fleksibilitas yang baik dan berpotensi untuk dikembangkan pada skala yang lebih besar di masa mendatang.

Link Github : https://github.com/aqilahakmaa/MQTT_SmartRoom_CPS_Monitoring